

MQTT 3.1.1 Paket Yapısı Referansı

Smart Bike IoT projesi için byte byte MQTT paket referansı

Hazırlayan: Hasan Basri Soylu Tarih: 2026-06

İçindekiler

1. Giriş
2. Tüm Paketlerin Ortak Yapısı
3. Remaining Length Encoding (VLE)
4. Paket Tipleri Özet
5. CONNECT
6. CONNACK
7. PUBLISH
8. PUBACK
9. SUBSCRIBE
10. SUBACK
11. UNSUBSCRIBE
12. UNSUBACK
13. PINGREQ / PINGRESP
14. DISCONNECT
15. QoS 2 Üçlüsü (PUBREC/REL/COMP)
16. Hızlı Referans Tablosu
17. Pratik Örnekler — Smart Bike IoT

1. Giriş

MQTT (Message Queuing Telemetry Transport), TCP/IP üzerinde çalışan hafif bir publish/subscribe mesajlaşma protokolüdür. IoT cihazlarda yaygın kullanılır çünkü:

- Düşük bant genişliği yeterli
- Az CPU/RAM gerektirir
- NAT/firewall arkasındaki cihazlar broker üzerinden konuşabilir
- Tek bir TCP bağlantısı üstünden çift yönlü mesajlaşma

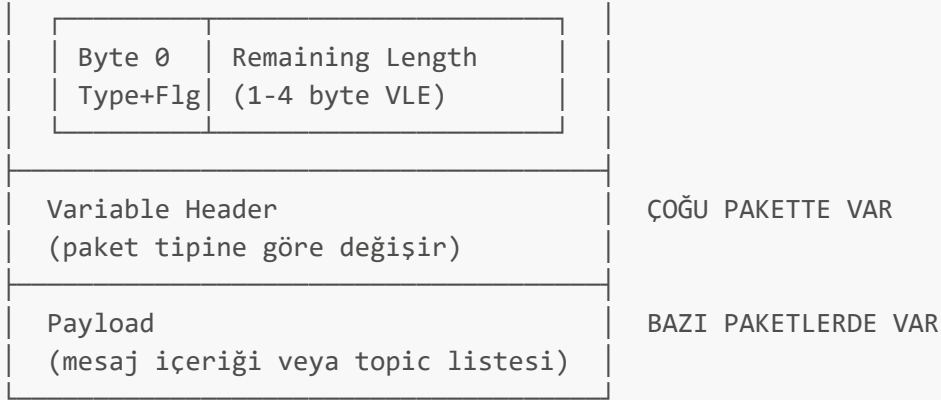
Bu doküman MQTT 3.1.1 protokolünün **paket byte yapılarını** referans olarak sunar.

2. Tüm Paketlerin Ortak Yapısı

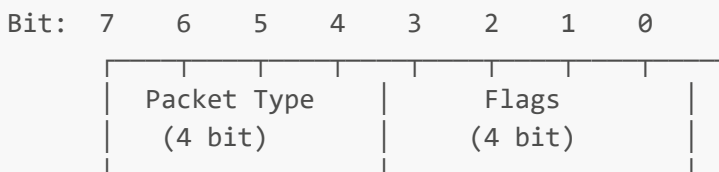
Her MQTT paketi şu üç bölümden oluşur:

Fixed Header

HER PAKETTE VAR



Kontrol byte (Byte 0) yapısı:



Üst 4 bit paket tipi, alt 4 bit flag (paket tipine göre anlamı değişir).

3. Remaining Length Encoding (VLE)

Remaining Length = Fixed Header'dan sonraki TÜM byte'ların toplamı.

Variable Length Encoding (VLE) kullanır — 1 ila 4 byte. Her byte'ın **MSB (en üst bit)** "devam ediyor" bayrağıdır.

Uzunluk Aralığı	Byte Sayısı	Maksimum Değer
0 — 127	1 byte	0x7F
128 — 16,383	2 byte	0xFF 0x7F
16,384 — 2,097,151	3 byte	0xFF 0xFF 0x7F
2,097,152 — 268,435,455	4 byte	0xFF 0xFF 0xFF 0x7F

Encode algoritması:

```
int mqtt_encode_remaining_length(uint8_t *buf, uint32_t length)
{
    int i = 0;
    do {
        uint8_t byte = length % 128;
        length /= 128;
        if (length > 0) {
            byte |= 0x80; // devam ediyor bayrağı
        }
        buf[i++] = byte;
    } while (length > 0);
}
```

```

    }
    buf[i++] = byte;
} while (length > 0);
return i; // kaç byte yazıldı
}

```

Decode algoritması:

```

int mqtt_decode_remaining_length(const uint8_t *buf, uint32_t *out_length)
{
    uint32_t multiplier = 1;
    uint32_t value      = 0;
    int      i          = 0;
    uint8_t  byte;

    do {
        byte = buf[i++];
        value += (byte & 0x7F) * multiplier;
        if (multiplier > 128*128*128) return -1; // overflow
        multiplier *= 128;
    } while ((byte & 0x80) != 0);

    *out_length = value;
    return i; // kaç byte okundu
}

```

Örnekler:

Decimal	Binary (alt 7 bit'ler)	VLE Bytes
0	0000000	0x00
64	1000000	0x40
127	1111111	0x7F
128	0000001 0000000	0x80 0x01
200	0000001 1001000	0xC8 0x01
16383	1111111 1111111	0xFF 0x7F

4. Paket Tipleri Özet

MQTT 3.1.1'de 14 farklı paket tipi vardır:

#	Tip	Kontrol Byte	Yön	Min Boyut	Açıklama
1	CONNECT	0x10	C→B	~14 byte	Bağlantı kurulumu
2	CONNACK	0x20	B→C	4 byte	Bağlantı cevabı

#	Tip	Kontrol Byte	Yön	Min Boyut	Açıklama
3	PUBLISH	0x30-0x3F	↔	değişken	Mesaj yayını
4	PUBACK	0x40	↔	4 byte	QoS=1 ack
5	PUBREC	0x50	↔	4 byte	QoS=2 step 1
6	PUBREL	0x62	↔	4 byte	QoS=2 step 2
7	PUBCOMP	0x70	↔	4 byte	QoS=2 step 3
8	SUBSCRIBE	0x82	C→B	değişken	Abonelik isteği
9	SUBACK	0x90	B→C	değişken	Abonelik cevabı
10	UNSUBSCRIBE	0xA2	C→B	değişken	Abonelik iptali
11	UNSUBACK	0xB0	B→C	4 byte	İptal cevabı
12	PINGREQ	0xC0	C→B	2 byte	Keep-alive ping
13	PINGRESP	0xD0	B→C	2 byte	Keep-alive pong
14	DISCONNECT	0xE0	C→B	2 byte	Bağlantı sonu

C = Client, B = Broker

5. CONNECT (Client → Broker)

İstemcinin broker'a "merhaba" deyip bağlantı parametrelerini bildirdiği paket.

Yapı

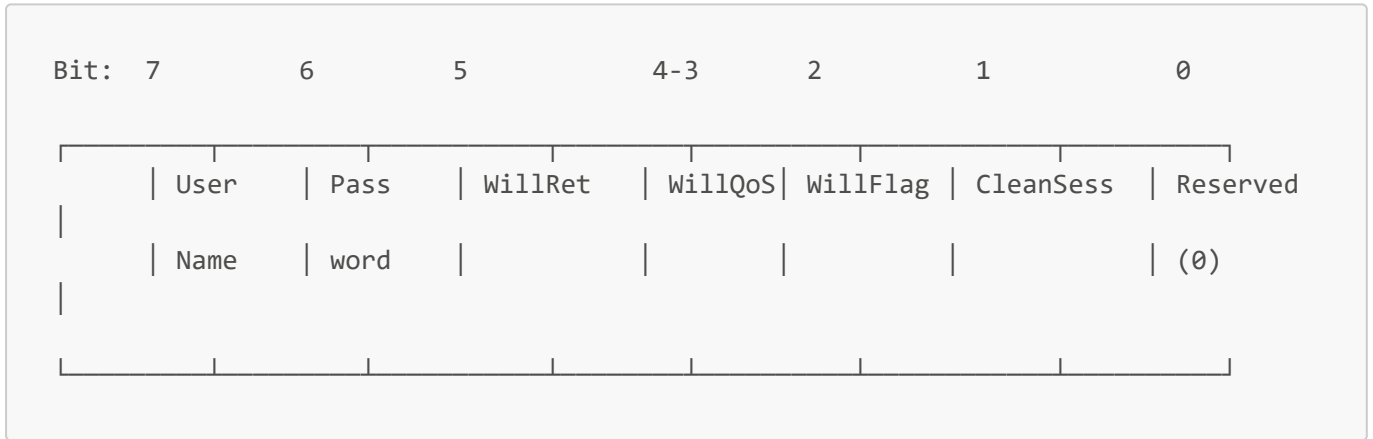
```
0x10 [rem_len] [protocol_name_len][protocol_name][level][flags][keep_alive]
      [client_id_len][client_id]
      [will_topic_len][will_topic]?
      [will_msg_len][will_msg]?
      [username_len][username]?
      [password_len][password]?
```

Byte tablosu

Byte	Field	Boyut	Açıklama	Örnek
0	Kontrol byte	1	0x10 = CONNECT	0x10
1	Remaining Length	1-4	VLE	0x16 (22)
Variable Header (Byte 2-11)				

Byte	Field	Boyut	Açıklama	Örnek
2-3	Protocol Name Length	2	Big-endian, 0x0004	0x00 0x04
4-7	Protocol Name	4	"MQTT"	'M' 'Q' 'T' 'T'
8	Protocol Level	1	0x04 = MQTT 3.1.1	0x04
9	Connect Flags	1	Bit alanı (aşağı bak)	0x02
10-11	Keep Alive	2	Saniye, big-endian	0x00 0x3C (60)
Payload				
12-13	Client ID Length	2	Big-endian	0x00 0x0A (10)
14+	Client ID	N	UTF-8 string	"esp32_bold"

Connect Flags (Byte 9) bit detayı



Bit	Anlam
7	User Name flag — payload'da username var mı
6	Password flag — payload'da password var mı
5	Will Retain — broker LWT'yi retained yapsın mı
4-3	Will QoS — LWT mesajının QoS'u
2	Will Flag — payload'da will topic + message var mı
1	Clean Session — 1 = oturum baştan başlasın
0	Reserved — 0 olmalı

Payload sırası (flag'lere göre)

1. Client ID (zorunlu)

2. Will Topic + Will Message (eğer Will Flag = 1)
3. Username (eğer User Name flag = 1)
4. Password (eğer Password flag = 1)

Bizim somut örnek

```
0x10 0x16
0x00 0x04 'M' 'Q' 'T' 'T'
0x04
0x02          ← CleanSession=1
0x00 0x3C          ← Keep-alive 60 saniye
0x00 0x0A
'e' 's' 'p' '3' '2' '_' 'b' 'o' 'l' 'd'
```

Toplam: 24 byte
Remaining length = 22 = 0x16

6. CONNACK (Broker → Client)

Broker'ın CONNECT'e cevabı. **Her zaman 4 byte sabit.**

Yapı

```
0x20 0x02 [session_present] [return_code]
```

Byte tablosu

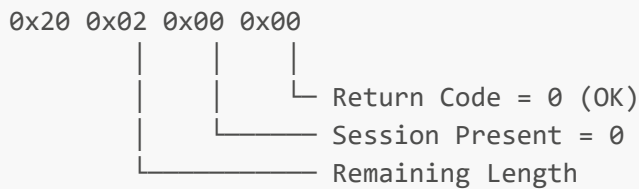
Byte	Field	Boyut	Açıklama
0	Kontrol byte	1	0x20 = CONNACK
1	Remaining Length	1	0x02 (her zaman)
2	Connect Acknowledge Flags	1	Bit 0: Session Present, gerisi reserved
3	Return Code	1	0x00=OK, gerisi hata

Return Code'lar

Kod	Anlam
0x00	Bağlantı kabul edildi
0x01	Protocol version desteklenmiyor
0x02	Client ID reddedildi
0x03	Broker müsait değil (server unavailable)

Kod	Anlam
0x04	Bad username veya password
0x05	Bağlanma yetkin yok (not authorized)

Örnek



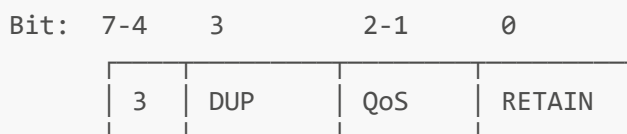
7. PUBLISH (Her iki yön)

Asıl mesaj. Bir topic'e payload yayını.

Yapı

```
0x3X [rem_len]
      [topic_len_msb][topic_len_lsb][topic]
      [packet_id_msb][packet_id_lsb]?    ← sadece QoS > 0
      [payload bytes...]
```

Kontrol byte (0x3X) bit detayı



Bit	Anlam
7-4	Packet Type = 3 (PUBLISH)
3	DUP — yeniden gönderim mi (QoS > 0 ile anlamlı)
2-1	QoS Level (0, 1, veya 2)
0	RETAIN — broker bunu son mesaj olarak saklasın mı

Yaygın kontrol byte değerleri

Hex	QoS	DUP	RETAIN
0x30	0	0	0
0x31	0	0	1
0x32	1	0	0
0x33	1	0	1
0x34	2	0	0
0x3A	1	1	0

Byte tablosu

Byte	Field	Boyut	Notlar
0	Kontrol byte	1	0x3X
1	Remaining Length	1-4	VLE
Variable Header			
2-3	Topic Length	2	Big-endian
4+	Topic	N	UTF-8, +/# YOK (publish'te wildcard olmaz)
(devam)	Packet ID	2	Sadece QoS > 0
Payload			
(devam)	Payload	M	İstediğin byte'lar

Bizim somut örnek

```

0x30          ← PUBLISH, QoS=0
0x67          ← Remaining length = 103
0x00 0x1E     ← Topic length = 30
'h' 'b' 's' '_' ... ← "hbs_smart_bike_2026_xyz123/gps" (30 byte)
'{' ' "' 'C' 'i' 'h' 'a' 'z' '_' 'I' 'D' ' "' ':' '4' '2' ',' ... ← JSON (71 byte)

Toplam: 105 byte

```

8. PUBACK

QoS=1 PUBLISH için cevap. **Her zaman 4 byte sabit.**

Yapı

```
0x40 0x02 [packet_id_msb] [packet_id_lsb]
```

Byte tablosu

Byte	Field	Boyut
0	Kontrol byte	0x40
1	Remaining Length	0x02
2-3	Packet ID	2 byte (PUBLISH'tekiyle aynı)

Packet ID ile hangi PUBLISH'in ack'i olduğunu eşleştirirsin.

9. SUBSCRIBE (Client → Broker)

Topic'lere abone ol.

Yapı

```
0x82 [rem_len]
    [packet_id_msb][packet_id_lsb]
    [topic_len_msb][topic_len_lsb][topic][qos]
    [topic_len_msb][topic_len_lsb][topic][qos] ← birden çok topic
    ...
```

Byte tablosu

Byte	Field	Boyut	Notlar
0	Kontrol byte	1	0x82 (alt 4 bit 0010 ZORUNLU)
1	Remaining Length	1-4	VLE
Variable Header			
2-3	Packet ID	2	SUBACK eşleştirmesi için
Payload (her topic için tekrar)			
-	Topic Filter Length	2	Big-endian
-	Topic Filter	N	UTF-8, + ve # wildcard'ları kullanılabilir
-	Requested QoS	1	0x00, 0x01, veya 0x02

Kontrol byte (0x82) neden?

Bit: 7-4 3-0

8	0010
---	------

Packet Type = 8 (SUBSCRIBE)

Alt 4 bit MUTLAKA 0010 → MQTT spec gereği

Eğer farklı yollasak broker bağlantıyı koparır.

Çoklu topic örneği

```
0x82
0x14          ← Remaining length = 20
0x00 0x01     ← Packet ID = 1
0x00 0x03 'a' '/' 'b' 0x00 ← topic "a/b", QoS 0
0x00 0x03 'c' '/' 'd' 0x01 ← topic "c/d", QoS 1
```

Topic Wildcard'ları

- **+** — tek seviye wildcard: **bisiklet/+/hiz** matches **bisiklet/123/hiz**, **bisiklet/456/hiz**
- **#** — çok seviye wildcard, sadece sonda: **bisiklet/#** matches her şey altında
- Sadece SUBSCRIBE'da kullanılır, PUBLISH'te asla

10. SUBACK (Broker → Client)

SUBSCRIBE'a cevap.

Yapı

```
0x90 [rem_len]
      [packet_id_msb][packet_id_lsb]
      [return_code_1][return_code_2]... ← SUBSCRIBE'daki her topic için 1 byte
```

Byte tablosu

Byte	Field	Boyut
0	Kontrol byte	0x90
1	Remaining Length	VLE
2-3	Packet ID	2 (SUBSCRIBE'daki ile aynı)
4+	Return Code'lar	N (her topic için 1 byte)

SUBACK Return Code'lar

Kod	Anlam
0x00	Granted QoS 0
0x01	Granted QoS 1
0x02	Granted QoS 2
0x80	Reddedildi (yetki yok, vs.)

Önemli: Broker istediğinden DÜŞÜK QoS verebilir. Mesela QoS 2 istedin, broker QoS 1 verebilir.

11. UNSUBSCRIBE

Aboneliği iptal et.

Yapı

```
0xA2 [rem_len]
      [packet_id_msb][packet_id_lsb]
      [topic_len_msb][topic_len_lsb][topic] ← QoS YOK (sub'tan farklı)
      ...
```

Byte tablosu

Byte	Field	Notlar
0	Kontrol byte	0xA2 (alt 4 bit 0010 zorunlu)
1	Remaining Length	VLE
2-3	Packet ID	
4+	Topic'ler	Her biri length + string, QoS YOK

12. UNSUBACK

UNSUBSCRIBE cevabı. **4 byte sabit.**

```
0xB0 0x02 [packet_id_msb] [packet_id_lsb]
```

Sadece packet ID eşleştirme, başka bilgi yok.

13. PINGREQ / PINGRESP

Keep-alive ping/pong. **Her ikisi 2 byte sabit.**

PINGREQ (Client → Broker)

0xC0 0x00

PINGRESP (Broker → Client)

0xD0 0x00

Ne zaman kullanılır?

CONNECT'te belirttiğin keep-alive süresi (örn. 60sn) içinde **hiçbir paket yollamazsan** broker bağlantıyı koparır. PINGREQ "hâlâ buradayım" der.

Pratik kural: Her keep-alive aralığının ~%80'inde PINGREQ yolla. 60sn keep-alive için her 48 saniyede bir.

Eğer zaten düzenli PUBLISH yapıyorsan PINGREQ gerekmez — PUBLISH de keep-alive timer'ı resetler.

14. DISCONNECT

Kibarca bağlantıyı kapat. **2 byte sabit.**

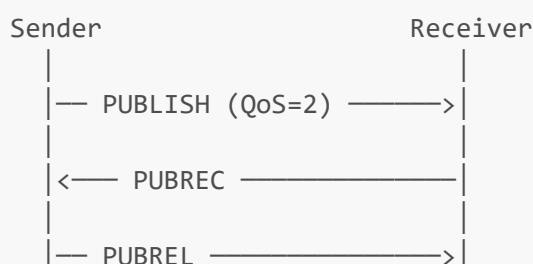
0xE0 0x00

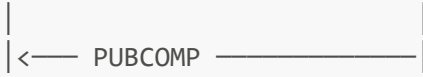
DISCONNECT yolladığında broker **Last Will Testament (LWT) mesajını tetiklemez** (kibar ayrılış). TCP koparsa veya DISCONNECT yollamadan dönersen LWT tetiklenir.

15. QoS 2 Üçlüsü (PUBREC/REL/COMP)

QoS=2 "tam 1 kez teslim" garantisi için 4-yönlü handshake.

Akış





Paket yapıları (hepsi 4 byte sabit)

```
PUBREC    0x50 0x02 [pkt_id_msb] [pkt_id_lsb]
PUBREL    0x62 0x02 [pkt_id_msb] [pkt_id_lsb] ← alt 4 bit 0010 ZORUNLU
PUBCOMP   0x70 0x02 [pkt_id_msb] [pkt_id_lsb]
```

QoS 2 nadiren kullanılır çünkü 4x bandwidth + latency cezası ağır.

16. Hızlı Referans Tablosu

Paket	İlk Byte	Min Boyut	Yön	Variable Header	Payload
CONNECT	0x10	~14 B	C→B	10 byte (prot + flags + keepalive)	Client ID + opsiyonel
CONNACK	0x20	4 B sabit	B→C	2 byte (flags + return code)	yok
PUBLISH	0x30–0x3F	değişken	↔	Topic + opsiyonel Packet ID	Mesaj
PUBACK	0x40	4 B sabit	↔	2 byte (Packet ID)	yok
PUBREC	0x50	4 B sabit	↔	2 byte (Packet ID)	yok
PUBREL	0x62	4 B sabit	↔	2 byte (Packet ID)	yok
PUBCOMP	0x70	4 B sabit	↔	2 byte (Packet ID)	yok
SUBSCRIBE	0x82	değişken	C→B	2 byte (Packet ID)	Topic + QoS listesi
SUBACK	0x90	değişken	B→C	2 byte (Packet ID)	Return Code listesi
UNSUBSCRIBE	0xA2	değişken	C→B	2 byte (Packet ID)	Topic listesi
UNSUBACK	0xB0	4 B sabit	B→C	2 byte (Packet ID)	yok
PINGREQ	0xC0	2 B sabit	C→B	yok	yok
PINGRESP	0xD0	2 B sabit	B→C	yok	yok
DISCONNECT	0xE0	2 B sabit	C→B	yok	yok

Önemli kontrol byte alt 4 bit kuralları

- **PUBLISH:** serbest — DUP/QoS/RETAIN flag'leri için
- **SUBSCRIBE:** 0010 zorunlu
- **UNSUBSCRIBE:** 0010 zorunlu

- **PUBREL:** 0010 zorunlu
- Diğerleri: 0000

17. Pratik Örnekler — Smart Bike IoT

Örnek 1: CONNECT paketi

Bizim test paketi (esp32_bold client ID):

```
static const uint8_t mqtt_connect[] = {
    0x10, 0x16, // CONNECT, len=22
    0x00, 0x04, 'M', 'Q', 'T', 'T', // Protocol name "MQTT"
    0x04, // Protocol level 4 (3.1.1)
    0x02, // CleanSession=1
    0x00, 0x3C, // Keep-alive 60sn
    0x00, 0x0A, // Client ID length 10
    'e', 's', 'p', '3', '2', '_', 'b', 'o', 'l', 'd'
};
// Toplam 24 byte
```

Örnek 2: PUBLISH paketi (struct → JSON → MQTT)

Topic: `hbs_smart_bike_2026_xyz123/gps` (30 byte) Payload: `{"Cihaz_ID":42,"Yukseklık":120.50,...}` (71 byte)

```
const char *topic      = "hbs_smart_bike_2026_xyz123/gps";
uint16_t   topic_len  = strlen(topic);
uint8_t    publish_pkt[256];
uint16_t   pkt_idx    = 0;

publish_pkt[pkt_idx++] = 0x30; // PUBLISH, QoS=0
publish_pkt[pkt_idx++] = (uint8_t)(2 + topic_len + json_len); // Remaining
length
publish_pkt[pkt_idx++] = (uint8_t)(topic_len >> 8); // Topic len MSB
publish_pkt[pkt_idx++] = (uint8_t)(topic_len & 0xFF); // Topic len LSB
memcpy(&publish_pkt[pkt_idx], topic, topic_len);
pkt_idx += topic_len;
memcpy(&publish_pkt[pkt_idx], json_payload, json_len);
pkt_idx += json_len;

// Toplam 105 byte
```

Örnek 3: SUBSCRIBE paketi

Topic: `hbs_smart_bike_2026_xyz123/komut` (32 byte), QoS=0

```

0x82          ← SUBSCRIBE
0x25          ← Remaining length = 37
0x00 0x01     ← Packet ID = 1
0x00 0x20     ← Topic length = 32
'h' 'b' 's' '_' 's' 'm' 'a' 'r' 't' '_' 'b' 'i' 'k' 'e' '_'
'2' '0' '2' '6' '_' 'x' 'y' 'z' '1' '2' '3' '/' 'k' 'o' 'm' 'u' 't'
0x00          ← Requested QoS = 0

```

Toplam: 39 byte

Örnek 4: SUBACK karşılığı (broker'dan gelen)

0x90 0x03 0x00 0x01 0x00

Granted QoS 0
Packet ID LSB
Packet ID MSB (1)
Remaining length 3

Örnek 5: PINGREQ / PINGRESP

```
// PINGREQ (gönder)
const uint8_t pingreq[] = { 0xC0, 0x00 };
sim800c_tcp_send(pingreq, 2);

// PINGRESP (al)
uint8_t pingresp[2];
sim800c_tcp_recv(pingresp, 2, 5000);
// pingresp = { 0xD0, 0x00 }
```

Örnek 6: DISCONNECT (kibar ayrılış)

```
const uint8_t disconnect[] = { 0xE0, 0x00 };
sim800c_tcp_send(disconnect, 2);
sim800c_tcp_close();
```

Bonus: MQTT Topic Naming Best Practices

1. **Hiyerarşik tasarım** kullan: `<organization>/<device_id>/<resource>`

- İyi: `hbs/bike_42/gps`, `hbs/bike_42/komut`
- Kötü: `data42`, `mybike_gps_v2` `real_final`

2. Aynı cihaza ait topic'leri ortak prefix ile grupla

- Pattern: `hbs/bike_42/+` ile o cihazın tüm topic'lerini dinleyebilirsin

3. Tahmin edilemez segment kullan (public broker'da)

- `bike_42` kolay tahmin edilir
- `bike_xyz789abc` daha güvenli

4. Leading slash kullanma: `/bike/42/gps` ❌, `bike/42/gps` ✅

5. Boşluk veya özel karakter kullanma: ASCII alphanumeric + `_` + `-` + `/` yeterli

6. `$` ile başlayan topic'ler broker özel: `$SYS/...` broker stats için, sen kullanma

Kaynaklar

- **MQTT 3.1.1 OASIS Specification:** <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- **HiveMQ MQTT Essentials:** <https://www.hivemq.com/mqtt-essentials/>
- **Eclipse Paho MQTT C/Embedded:** <https://github.com/eclipse/paho.mqtt.embedded-c>
- **MQTT-C (single-header library):** <https://github.com/LiamBindle/MQTT-C>

Doküman sonu

Bu doküman MQTT 3.1.1 odaklıdır. MQTT 5.0 için ekstra paket (AUTH `0xF0`) ve "properties" alanları vardır — ileride incelenebilir.